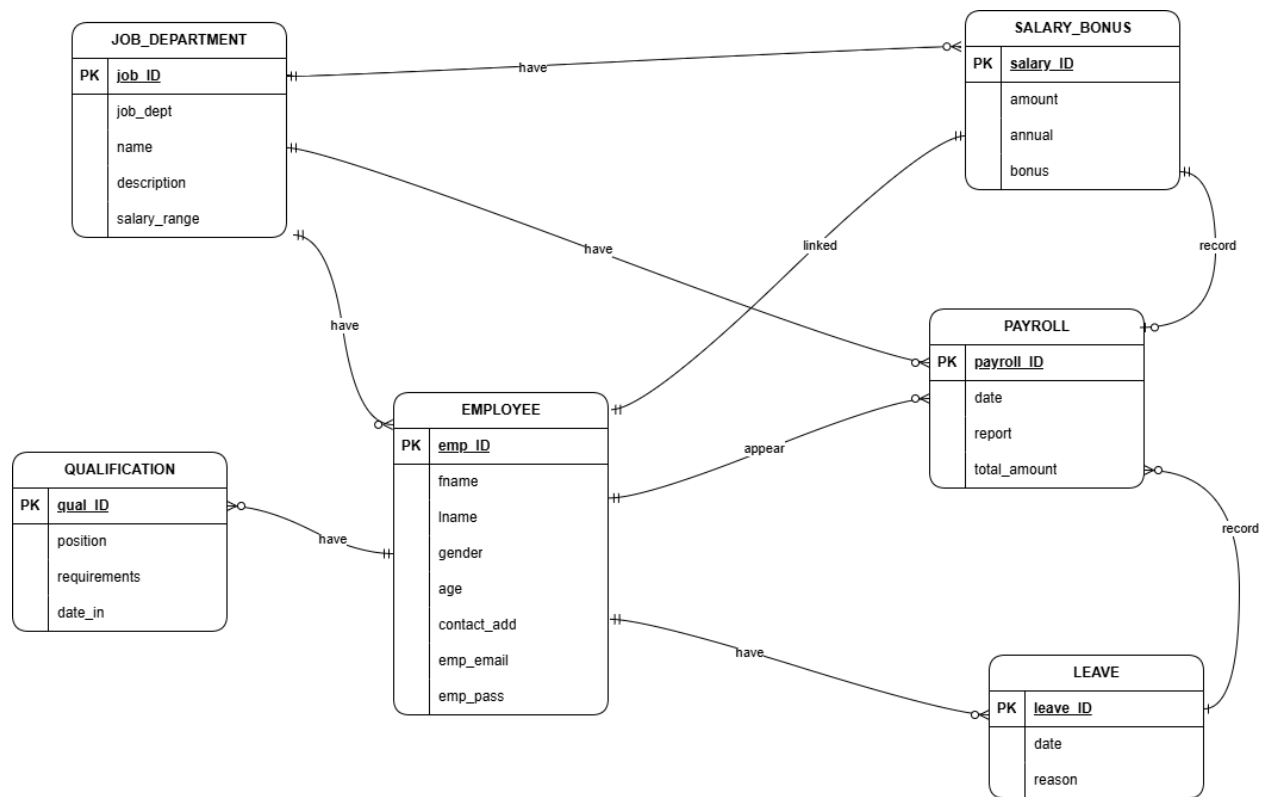


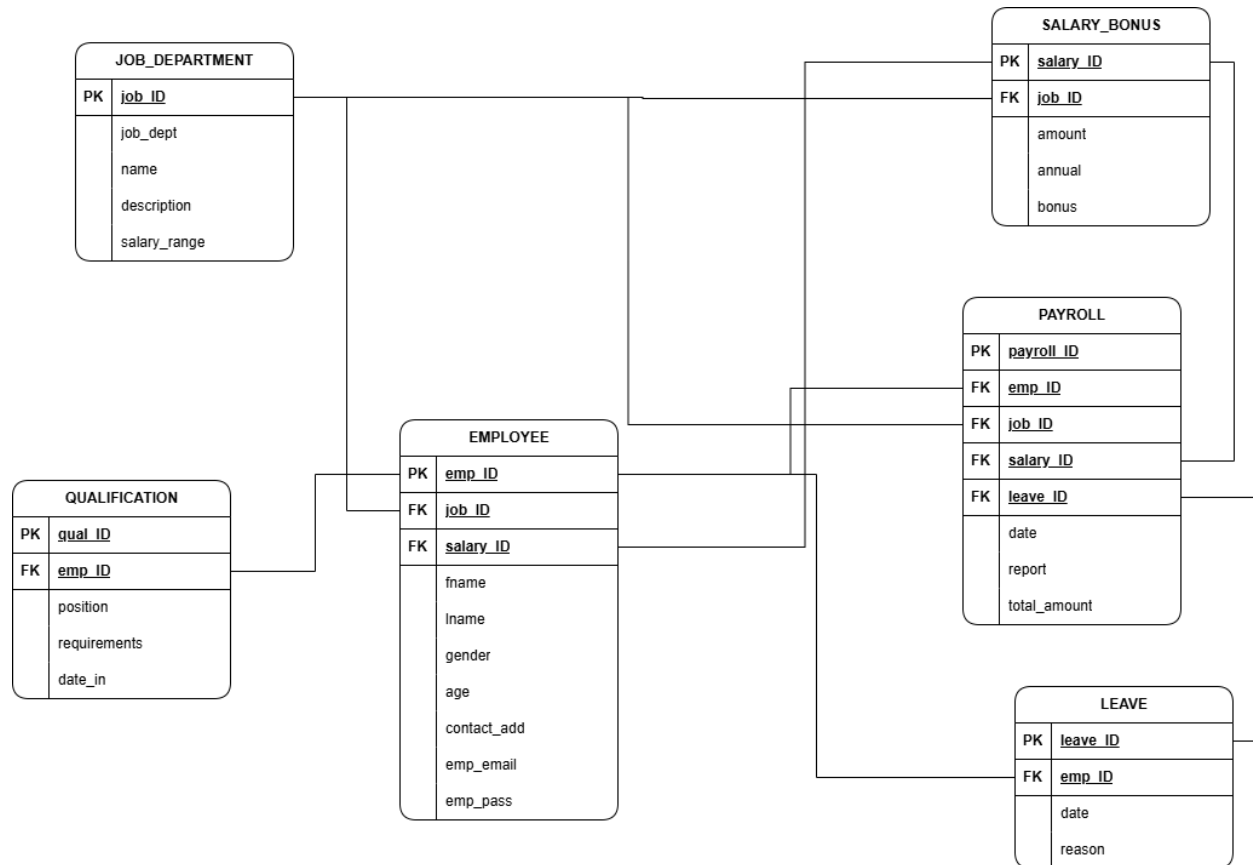
EMPLOYEE MANAGEMENT SYSTEM

01 ERD Design & Mapping

Task 1: Draw the Full ERD



Task 2: Map the ERD to a Relational Schema



EMS Schema

JOB_DEPARTMENT (**job_ID**, job_dept, name, description, salary_range)

SALARY_BONUS (**salary_ID**, amount, annual, bonus, job_ID)

EMPLOYEE (**emp_ID**, fname, lname, gender, age, contact_add, emp_email, emp_pass, job_ID, salary_ID)

QUALIFICATION (**qual_ID**, position, requirements, date_in, emp_ID)

LEAVE (**leave_ID**, date, reason, emp_ID)

PAYROLL (**payroll_ID**, date, report, total_amount, emp_ID, job_ID, salary_ID, leave_ID)

02 Normalization

Task 1: First Normal Form (1NF) Analysis

Rule: every cell must hold exactly one atomic value. No lists, no repeating groups, no multivalued columns.

Table	1NF Rule Applied
JOB_DEPARTMENT	job_dept, name, description, salary_range — each attribute holds one atomic value. No column stores a list.
SALARY_BONUS	amount, annual, bonus — each is a single atomic numeric value.
EMPLOYEE	fname, lname, gender, age, contact_add, emp_email, emp_pass — every attribute is atomic. No column stores multiple emails, phone numbers, or values.
QUALIFICATION	position, requirements, date_in — atomic. An employee with multiple qualifications is represented as multiple ROWS (via the emp_ID FK), not as multiple values packed into one cell.
LEAVE	date, reason — atomic. Multiple leave records per employee = multiple rows, not a list in one cell.
PAYROLL	date, report, total_amount — atomic. Each payroll entry is one row with single-valued fields.

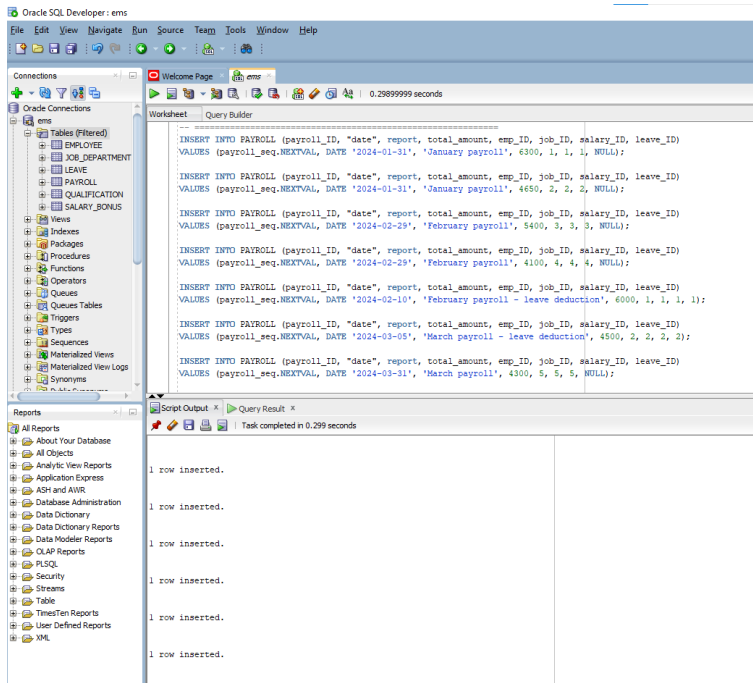
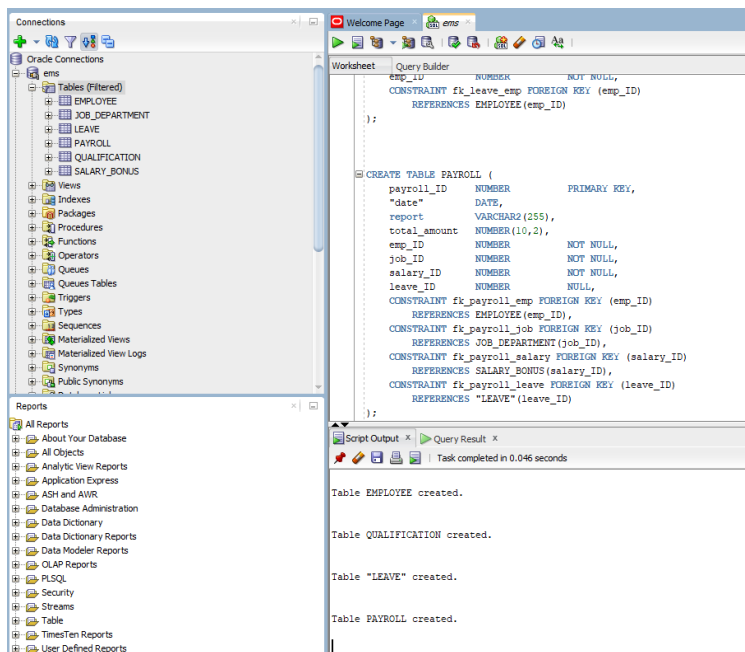
Task 2: Second Normal Form (2NF) Analysis

Rule: a table must already be in 1NF, and every non-key attribute must depend on the **WHOLE** primary key — not just part of it. This rule only applies when a table has a composite (multi-column) primary key. If a table's primary key is a single column, it automatically satisfies 2NF once it is in 1NF.

Table	Primary Key	Composite PK?	2NF Result
JOB_DEPARTMENT	job_ID	No	Satisfies 2NF
SALARY_BONUS	salary_ID	No	Satisfies 2NF
EMPLOYEE	emp_ID	No	Satisfies 2NF
QUALIFICATION	qual_ID	No	Satisfies 2NF
LEAVE	leave_ID	No	Satisfies 2NF
PAYROLL	payroll_ID	No	Satisfies 2NF

03 SQL DML — INSERT / UPDATE / DELETE / SELECT

Task 1: Populate All Tables with Seed Data



Task 2: Conditional SELECT Queries

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema with tables like EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, and SALARY_BONUS. The main workspace contains a SQL query and its results.

Query:

```
-- (c) Find all employees who have taken leave with reason
-- containing the word "sick" (case-insensitive).
SELECT e.emp_id,
       e.fname || ' ' || e.lname AS employee_name,
       l.reason,
       l."date" AS leave_date
FROM EMPLOYEE e
JOIN "LEAVE" l ON e.emp_id = l.emp_id
WHERE LOWER(l.reason) LIKE 'sick%';
```

Query Result:

EMP_ID	NAME	AGE
25	Rida Abdallah	36
2	Sara Ahmed	34
5	Yusif Hamed	38
24	Ali Mahmoud	33
21	Layla Hasser	31
23	Fatma Rashid	27
4	Maya Said	26
1	John Smith	29

0 rows selected.

Query Result:

PAYROLL_ID	EMPLOYEE_NAME	DEPARTMENT_NAME	TOTAL_AMOUNT	PAYROLL_D
1	John Smith	Information Technology	6300	31-JAN-24
5	John Smith	Information Technology	6000	10-FEB-24
3	Omar Khalid	Finance	5400	29-FEB-24
8	Omar Khalid	Finance	5200	16-APR-24

Query Result:

EMP_ID	EMPLOYEE_NAME	REASON
1	John Smith	Sick leave - flu
4	Maya Said	Sick leave - fever

04 Aggregation Functions

Task 1: Basic Aggregation

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema with tables EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, and SALARY_BONUS. The main pane shows a query with three parts: (a) Total number of employees in each department, (b) Minimum, maximum, and average salary (amount) across all salary records, and (c) Total bonus paid out across the entire company. The results are displayed in a table format.

```
-- (a) Total number of employees in each department.
SELECT d.name AS department_name,
COUNT(e.emp_id) AS total_employees
FROM JOB_DEPARTMENT d
JOIN EMPLOYEE e ON e.job_id = d.job_id
GROUP BY d.name;

-- (b) Minimum, maximum, and average salary (amount) across
-- all salary records.
SELECT MIN(amount) AS min_salary,
MAX(amount) AS max_salary,
ROUND(AVG(amount), 2) AS avg_salary
FROM SALARY_BONUS;

-- (c) Total bonus paid out across the entire company.
SELECT SUM(bonus) AS total_bonus_paid
FROM SALARY_BONUS;
```

DEPARTMENT_NAME	TOTAL_EMPLOYEES
Sales	2
Information Technology	2
Finance	2
Human Resources	2
Operations	2

MIN_SALARY	MAX_SALARY	AVG_SALARY
40000	61000	48300

TOTAL_BONUS_PAID
18000

Task 2: GROUP BY with HAVING

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema. The main pane shows a query with three parts: (a) List departments where the average employee age exceeds 30, (b) Show all job titles (QUALIFICATION, position) where more than 2 employees share that qualification position, and (c) Find months (from PAYROLL, "date") where the total payroll amount exceeds 20,000. The results are displayed in a table format.

```
-- (a) List departments where the average employee age
-- exceeds 30.
SELECT d.name AS department_name,
ROUND(AVG(e.age), 2) AS avg_age
FROM JOB_DEPARTMENT d
JOIN EMPLOYEE e ON e.job_id = d.job_id
GROUP BY d.name
HAVING AVG(e.age) > 30;

-- (b) Show all job titles (QUALIFICATION, position) where more
-- than 2 employees share that qualification position.
SELECT q.position,
COUNT(q.emp_id) AS employee_count
FROM QUALIFICATION q
GROUP BY q.position
HAVING COUNT(q.emp_id) > 2;

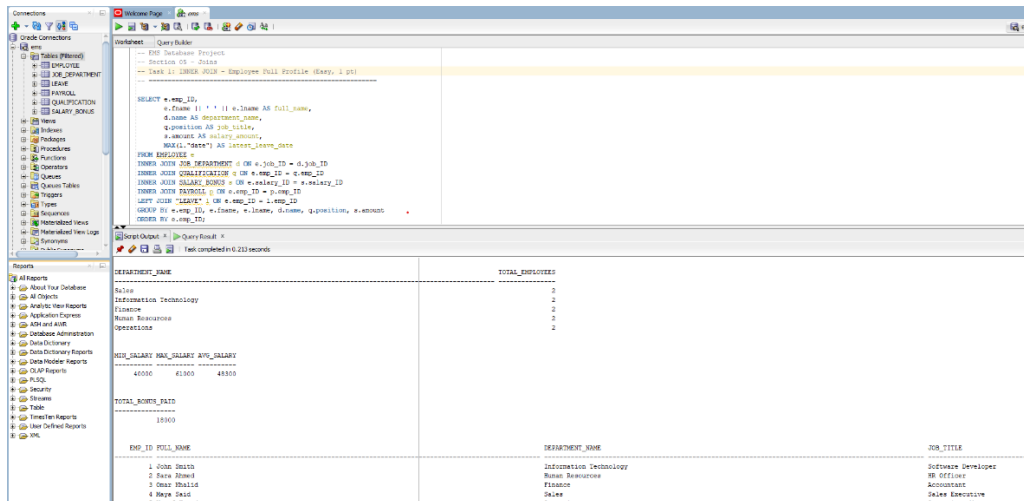
-- (c) Find months (from PAYROLL, "date") where the total
-- payroll amount exceeds 20,000.
SELECT TO_CHAR(p, "date", 'YYYY-MM') AS payroll_month,
SUM(p.total_amount) AS total_payroll
FROM PAYROLL p
GROUP BY TO_CHAR(p, "date", 'YYYY-MM')
HAVING SUM(p.total_amount) > 20000;
```

DEPARTMENT_NAME	AVG_AGE
Finance	34
Human Resources	39.5
Operations	37

no rows selected
no rows selected

05 Joins

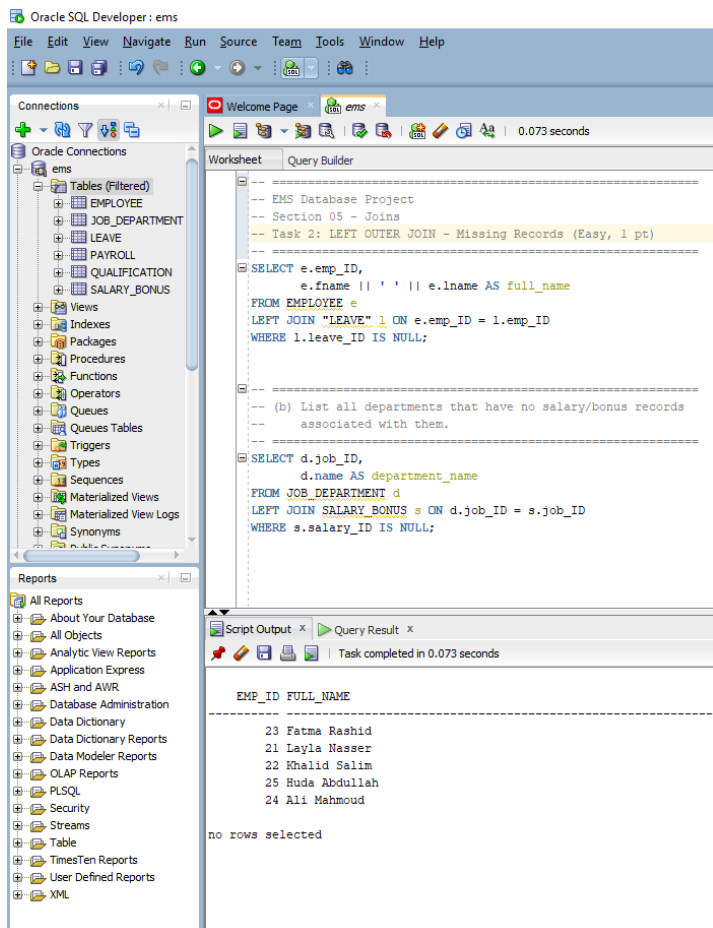
Task 1: INNER JOIN — Employee Full Profile



The screenshot shows the SQL Developer interface with a query window titled 'Query Builder'. The query is an inner join between the EMPLOYEE and LEAVE tables. The results pane shows a table with columns: DEPARTMENT_NAME, TOTAL_EMPLOYEES, and EMP_ID. The data is as follows:

DEPARTMENT_NAME	TOTAL_EMPLOYEES	EMP_ID
Research and Development	14	1
Marketing	5	2
Finance	3	3
Human Resources	2	4
Operations	2	5

Task 2: LEFT OUTER JOIN — Missing Records



The screenshot shows the SQL Developer interface with a query window titled 'Query Builder'. The query is a left outer join between the EMPLOYEE and LEAVE tables. The results pane shows a table with columns: EMP_ID, FULL_NAME, and LEAVE_ID. The data is as follows:

EMP_ID	FULL_NAME	LEAVE_ID
23	Fatma Rashid	
21	Layla Nasser	
22	Khalid Salim	
25	Huda Abdullah	
24	Ali Mahmoud	

06 Subqueries

Task 1: Single-Row Subquery

Oracle SQL Developer: ems

Connections: ems

Tables (Filtered): EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, SALARY_BONUS

Views: Views, Indexes, Packages, Procedures, Functions, Operators, Queues, Queues Tables, Triggers, Types, Sequences, Materialized Views, Materialized View Logs, Synonyms

Reports: All Reports, About Your Database, All Objects, Analytic View Reports, Application Express, ASH and AWR, Database Administration, Data Dictionary, Data Dictionary Reports, Data Modeler Reports, OLAP Reports, PLSQL, Security, Streams, Table, TimesTen Reports, User Defined Reports, XML

Script Output: Task completed in 0.083 seconds

Query:

```
-- (e) Find all employees whose salary is greater than the
-- average salary of the entire company.
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS full_name,
       s.amount AS salary_amount
FROM EMPLOYEE e
JOIN SALARY_BONUS s ON e.salary_ID = s.salary_ID
WHERE s.amount > (
    SELECT AVG(amount) FROM SALARY_BONUS
);

-- (d) Retrieve the department with the highest total payroll
-- amount.
SELECT d.job_ID,
       d.name AS department_name
FROM JOB_DEPARTMENT d
WHERE d.job_ID = (
    SELECT p.job_ID
    FROM PAYROLL p
    GROUP BY p.job_ID
)
```

EMP_ID	FULL_NAME	SALARY_AMOUNT
1	John Smith	60000
3	Omar Khalid	52000
21	Layla Hasser	61000
23	Fatma Rashid	53000

JOB_ID	DEPARTMENT_NAME
1	Information Technology

Task 2: Multi-Row Subquery with IN / ANY / ALL

Oracle SQL Developer: ems

Connections: ems

Tables (Filtered): EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, SALARY_BONUS

Views: Views, Indexes, Packages, Procedures, Functions, Operators, Queues, Queues Tables, Triggers, Types, Sequences, Materialized Views, Materialized View Logs, Synonyms

Reports: All Reports, About Your Database, All Objects, Analytic View Reports, Application Express, ASH and AWR, Database Administration, Data Dictionary, Data Dictionary Reports, Data Modeler Reports, OLAP Reports, PLSQL, Security, Streams, Table, TimesTen Reports, User Defined Reports, XML

Script Output: Task completed in 0.139 seconds

Query:

```
FROM EMPLOYEE e
JOIN SALARY_BONUS s ON e.salary_ID = s.salary_ID
WHERE s.amount > ALL (
    SELECT s2.amount
    FROM SALARY_BONUS s2
    JOIN JOB_DEPARTMENT d ON s2.job_ID = d.job_ID
    WHERE d.name = 'Sales'
);

-- (c) Find employees whose salary is greater than ANY salary
-- in the 'HR' department. (Use ANY)
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS full_name,
       s.amount AS salary_amount
FROM EMPLOYEE e
JOIN SALARY_BONUS s ON e.salary_ID = s.salary_ID
WHERE s.amount > ANY (
    SELECT s2.amount
    FROM SALARY_BONUS s2
    JOIN JOB_DEPARTMENT d ON s2.job_ID = d.job_ID
    WHERE d.name = 'Human Resources'
);
```

EMP_ID	FULL_NAME	JOB_ID
1	John Smith	1
21	Layla Hasser	1
2	Sara Ahmed	2
22	Khalid Salim	2
3	Omar Khalid	3
23	Fatma Rashid	3
4	Maysa Said	4
24	Ali Mahamoud	4
5	Yusuf Hamed	5
25	Huda Abdullah	5

10 rows selected.

EMP_ID	FULL_NAME	SALARY_AMOUNT
1	John Smith	60000
2	Sara Ahmed	45000

07 Views

Task 1: Simple Read-Only View

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema with tables EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, and SALARY_BONUS. The main workspace shows the following SQL script:

```
-- Task 1: Simple Read-Only View (Easy, 1 pt)

-- Create or replace view VU_EMPLOYEE_SUMMARY AS
SELECT e.emp_id,
       e.fname || ' ' || e.lname AS full_name,
       e.gender,
       e.age,
       d.name AS department_name,
       g.position AS job_title
FROM EMPLOYEE e
JOIN JOB_DEPARTMENT d ON e.job_id = d.job_id
LEFT JOIN QUALIFICATION q ON e.emp_id = q.emp_id;

-- (a) Query the view to list all female employees over 30.
--
-- SELECT *
-- FROM VU_EMPLOYEE_SUMMARY
-- WHERE gender = 'F'
-- AND age > 30;

-- (b) Try to INSERT a row through this view and document the
```

The query results show the view was created successfully. The output table has columns EMP_ID, FULL_NAME, G, AGE, DEPARTMENT_NAME, and JOB_TITLE. The data is as follows:

EMP_ID	FULL_NAME	G	AGE	DEPARTMENT_NAME	JOB_TITLE
2	Sara Ahmed	F	34	Human Resources	HR Officer
21	Ismael Hassan	F	31	Information Technology	
25	Rada Abdulrah	F	36	Operations	

An error message is displayed at the bottom: "Error starting at line : 47 in command - INSERT INTO VU_EMPLOYEE_SUMMARY (emp_id, full_name, gender, age, department_name, job_title) VALUES (999, 'Test Person', 'M', 40, 'IT', 'Tester') Error at Command line : 47 Column : 34 Error report - SQL Error 904-01794: cannot modify a column which maps to a non key-preserved table".

Task 2: Payroll Dashboard View

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema with tables EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, and SALARY_BONUS. The main workspace shows the following SQL script:

```
-- Create or replace view VU_PAYROLL_DASHBOARD AS
SELECT p.payroll_id,
       e.fname || ' ' || e.lname AS employee_name,
       d.name AS department_name,
       s.amount AS salary_amount,
       s.bonus,
       l.reason AS leave_reason,
       p.date AS payroll_date,
       p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e ON p.emp_id = e.emp_id
JOIN JOB_DEPARTMENT d ON p.job_id = d.job_id
JOIN SALARY_BONUS s ON p.salary_id = s.salary_id
LEFT JOIN LEAVE l ON p.leave_id = l.leave_id;

-- Query the view to find the top 5 payroll records by
-- total amount.
--
-- SELECT *
-- FROM VU_PAYROLL_DASHBOARD
-- ORDER BY total_amount DESC
-- FETCH FIRST 5 ROWS ONLY;
```

The query results show the view was created successfully. The output table has columns PAYROLL_ID, EMPLOYEE_NAME, DEPARTMENT_NAME, SALARY_AMOUNT, BONUS, LEAVE, and PAYROLL_DATE. The data is as follows:

PAYROLL_ID	EMPLOYEE_NAME	DEPARTMENT_NAME	SALARY_AMOUNT	BONUS	LEAVE	PAYROLL_DATE
1	John Smith	Information Technology	60000	3000		
5	John Smith	Information Technology	60000	3000	Sick	
3	Omar Khalid	Finance	52000	2000		
8	Omar Khalid	Finance	52000	2000	Fam	
2	Sara Ahmed	Human Resources	45000	1500		

08 Stored Procedures & Functions

Task 1: Procedure — Add New Employee

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema with tables like EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, and SALARY_BONUS. The main workspace shows a SQL script for the procedure SP_ADD_EMPLOYEE. The script includes a cursor to get the next salary ID, inserts a new employee record, and a verification query. The bottom pane shows the execution results, indicating that the procedure compiled successfully and one row was inserted.

```
SQL Script: SP_ADD_EMPLOYEE.sql  
v_new_salary_id NUMBER;  
BEGIN  
  SELECT MAX(salary_id) INTO v_new_salary_id FROM SALARY_BONUS;  
  
  SP_ADD_EMPLOYEE(  
    p_fname => 'Nora',  
    p_lname => 'Saleh',  
    p_gender => 'F',  
    p_age => 28,  
    p_contact_add => '9 New St, Muscat',  
    p_emp_email => 'nora.saleh@ems.com',  
    p_emp_pass => 'pass123',  
    p_job_id => 1,  
    p_salary_id => v_new_salary_id  
  );  
END;  
/  
  
-- VERIFY: confirm only ONE 'Nora Saleh' row exists  
SELECT emp_id, fname, lname, emp_email  
FROM EMPLOYEE  
WHERE emp_email = 'nora.saleh@ems.com';
```

Script Output: Task completed in 0.495 seconds

Procedure SP_ADD_EMPLOYEE compiled

1 row inserted.

Commit complete.

SUCCESS: Employee Nora Saleh added with email nora.saleh@ems.com.

PL/SQL procedure successfully completed.

ERROR: An employee with email nora.saleh@ems.com already exists. Employee not added.

PL/SQL procedure successfully completed.

Task 2: Function — Calculate Net Salary

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema. The main workspace shows a SQL script for the function FN_NET_SALARY. The function calculates the net salary by subtracting bonuses from the salary. The bottom pane shows the execution results, displaying a table with employee IDs, full names, and net salaries.

```
SQL Script: FN_NET_SALARY.sql  
CREATE OR REPLACE FUNCTION FN_NET_SALARY (  
  p_emp_id IN EMPLOYEE.emp_id%TYPE  
) RETURN NUMBER  
IS  
  v_net_salary NUMBER;  
BEGIN  
  SELECT s.amount + s.bonus  
  INTO v_net_salary  
  FROM EMPLOYEE e  
  JOIN SALARY_BONUS s ON e.salary_id = s.salary_id  
  WHERE e.emp_id = p_emp_id;  
  
  RETURN v_net_salary;  
  
EXCEPTION  
  WHEN NO_DATA_FOUND THEN  
    RETURN NULL;  
END FN_NET_SALARY;  
/  
  
SELECT emp_id,  
       fname || ' ' || lname AS full_name,  
       FN_NET_SALARY(emp_id) AS net_salary  
FROM EMPLOYEE
```

Script Output: Task completed in 0.12 seconds

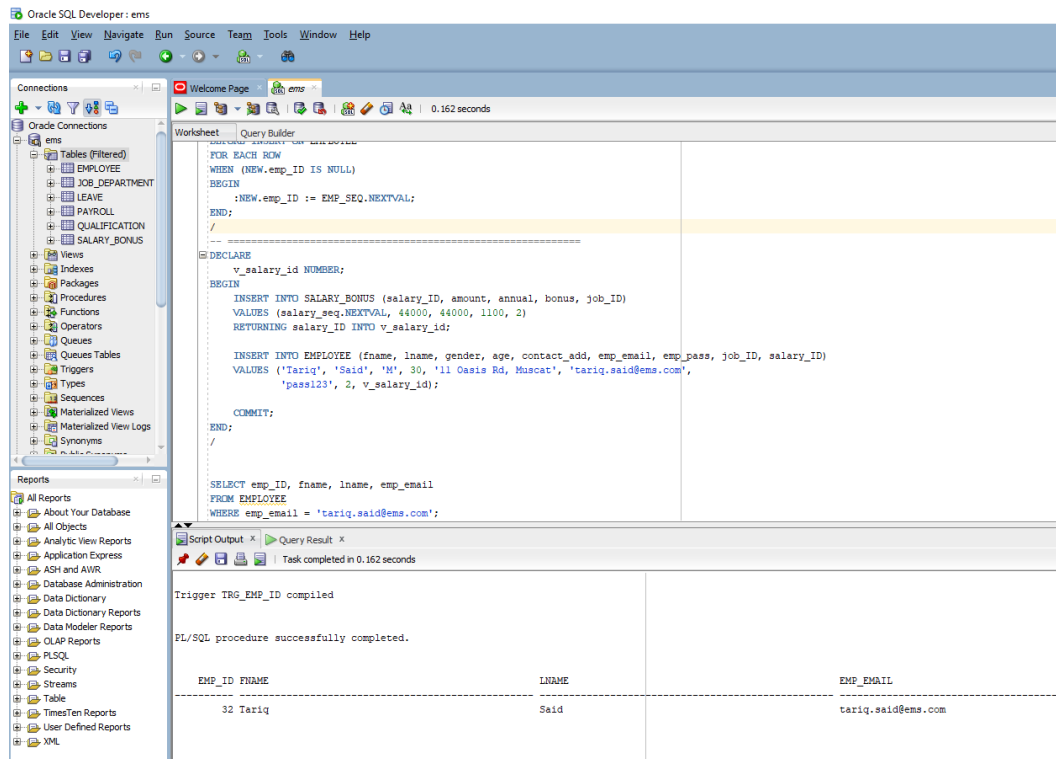
Function FN_NET_SALARY compiled

EMP_ID	FULL_NAME	NET_SALARY
21	Layla Nasser	64200
1	John Smith	63000
23	Fatma Rashid	58100
3	Omar Khalid	54000
31	Nora Saleh	48400
22	Khalid Salim	47600
2	Sara Ahmed	46500
25	Huda Abdullah	44300
5	Yusuf Hamed	43200
24	Ali Mahmoud	42100
4	Maya Said	41000

11 rows selected.

09 Triggers

Task 1: BEFORE INSERT — Auto-Assign emp ID



Oracle SQL Developer: ems

Connections: ems

Tables (Filtered): EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, SALARY_BONUS

Views: Views, Packages, Procedures, Functions, Operators, Queues, Queues Tables, Triggers, Types, Sequences, Materialized Views, Materialized View Logs, Synonyms

Reports: All Reports, About Your Database, All Objects, Analytic View Reports, Application Express, ASH and AWR, Database Administration, Data Dictionary, Data Dictionary Reports, Data Modeler Reports, OLAP Reports, PL/SQL, Security, Streams, Table, TimesTen Reports, User Defined Reports, XML

Script Output: X Query Result X

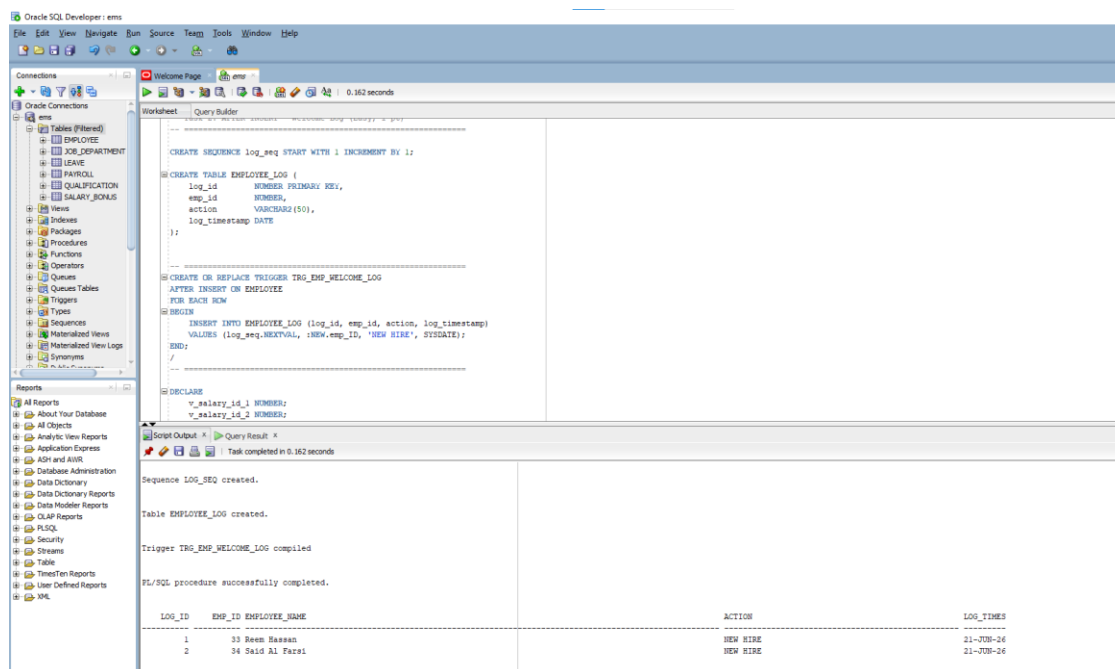
Task completed in 0.162 seconds

Trigger TRG_EMP_ID compiled

PL/SQL procedure successfully completed.

EMP_ID	FNAME	LNAME	EMP_EMAIL
32	Tariq	Said	tariq.said@ems.com

Task 2: AFTER INSERT — Welcome Log



Oracle SQL Developer: ems

Connections: ems

Tables (Filtered): EMPLOYEE, JOB_DEPARTMENT, LEAVE, PAYROLL, QUALIFICATION, SALARY_BONUS

Views: Views, Packages, Procedures, Functions, Operators, Queues, Queues Tables, Triggers, Types, Sequences, Materialized Views, Materialized View Logs, Synonyms

Reports: All Reports, About Your Database, All Objects, Analytic View Reports, Application Express, ASH and AWR, Database Administration, Data Dictionary, Data Dictionary Reports, Data Modeler Reports, OLAP Reports, PL/SQL, Security, Streams, Table, TimesTen Reports, User Defined Reports, XML

Script Output: X Query Result X

Task completed in 0.162 seconds

Sequence LOG_SEQ created.

Table EMPLOYEE_LOG created.

Trigger TRG_EMP_WELCOME_LOG compiled

PL/SQL procedure successfully completed.

LOG_ID	EMP_ID	EMPLOYEE_NAME	ACTION	LOG_TIMES
1	33	Reem Rayan	NEW HIRE	21-JUN-24
2	34	Said Al Farai	NEW HIRE	21-JUN-24

10 Oracle Scheduler Jobs

Task 1: Simple One-Time Scheduler Job

The screenshot shows the Oracle SQL Developer interface. On the left, the 'Connections' pane shows the 'ems' connection. The 'Tables (Filtered)' pane lists various database objects. The main workspace displays a PL/SQL script in the 'Query Builder' tab. The script creates a job named 'JOB_GREET_EMPLOYEES' using the DBMS_SCHEDULER package. The job is configured to run a PL/SQL block that outputs a message and inserts a log entry into the EMPLOYEE_LOG table. The job is set to run at a specific time and is enabled. Below the script, the 'Script Output' pane shows the message 'PL/SQL procedure successfully completed.' and 'no rows selected'.

```
DBMS_SCHEDULER.CREATE_JOB(  
  job_name      => 'JOB_GREET_EMPLOYEES',  
  job_type      => 'PLSQL_BLOCK',  
  job_action    => 'BEGIN  
    DBMS_OUTPUT.PUT_LINE('Payroll System Initialized');  
    INSERT INTO EMPLOYEE_LOG (log_id, emp_id, action, log_timestamp)  
    VALUES (log_seq.NEXTVAL, NULL, 'Payroll System Initialized', SYSDATE);  
    COMMIT;  
  END;',  
  start_date    => SYSTIMESTAMP + INTERVAL '2' MINUTE,  
  enabled       => TRUE  
);  
/  
  
--  
SELECT log_id, job_name, status, log_date  
FROM USER_SCHEDULER_JOB_LOG  
WHERE job_name = 'JOB_GREET_EMPLOYEES'  
ORDER BY log_date DESC;
```

Task 2: Recurring Job — Daily Leave Report

The screenshot shows the Oracle SQL Developer interface. On the left, the 'Connections' pane shows the 'ems' connection. The 'Tables (Filtered)' pane lists various database objects. The main workspace displays a PL/SQL script in the 'Query Builder' tab. The script creates a job named 'JOB_DAILY_LEAVE_REPORT' using the DBMS_SCHEDULER package. The job is configured to run a PL/SQL block that calculates the number of leave days for each employee and inserts a log entry into the EMPLOYEE_LOG table. The job is set to run daily at a specific time and is enabled. Below the script, the 'Script Output' pane shows the message 'PL/SQL procedure successfully completed.' and 'no rows selected'.

```
--  
-- DBMS Database Project  
-- Section 10 - Oracle Scheduler Jobs  
-- Task 2: Recurring Job - Daily Leave Report (Easy, 1 pt)  
  
BEGIN  
  DBMS_SCHEDULER.CREATE_JOB(  
    job_name      => 'JOB_DAILY_LEAVE_REPORT',  
    job_type      => 'PLSQL_BLOCK',  
    job_action    => 'DECLARE  
      v_leave_count NUMBER;  
  
      BEGIN  
        SELECT COUNT(*) INTO v_leave_count  
        FROM "LEAVE"  
        WHERE TRUNC("date") = TRUNC(SYSDATE);  
  
        INSERT INTO EMPLOYEE_LOG (log_id, emp_id, action, log_timestamp)  
        VALUES (log_seq.NEXTVAL, NULL,  
          'Daily Leave Report: ' || v_leave_count || ' leave record(s) today',  
          SYSDATE);  
  
        COMMIT;  
      END;',  
    start_date    => SYSTIMESTAMP,  
    repeat_interval => 'FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0',  
    enabled       => TRUE  
  );  
END;
```